Sonic Assistant (SAAPP): Architecture, Plumbing and History

Technical reference for Sonic Assistant, a product of Sonic Graph Technologies, built by Jack Harper. Last updated October 17, 2026. This document describes the system as it exists today, and a short history of how it got here.

1. What Sonic Assistant is

Sonic Assistant (SAAPP) is a multi-tenant AI assistant. It began in June 2026 as a retrieval-augmented generation (RAG) tool that answered questions only from documents in a knowledge base. It is now a personal agent that also remembers the people it talks to, uses tools (GitHub, Google Calendar, Gmail, Drive and Docs, a Python sandbox, a headless browser, the user's own local folder), writes documents with the user's approval, and takes care to behave well when someone is struggling. It is a full-stack cloud application: a React frontend, a FastAPI backend, MongoDB Atlas for data and vector search, and Google Gemini models.

Three ideas shape the design. First, every answer is grounded: from the knowledge base, from a tool result, or from memory, and the model is told which. Second, data is isolated per tenant: a user can only retrieve from the knowledge bases they are authorised for, enforced in the database query itself. Third, anything that changes the outside world (a pull request, an email, a calendar event, a file edit, a database write) is proposed first and only happens after the user approves it.

2. The stack at a glance

Layer	What it is
Frontend	React 19 and TypeScript, built with Vite, hash-based routing. Deployed on Vercel at sonicassistant.com. Light and dark themes. A mascot named Patchy reflects what the assistant is doing.
Backend API	FastAPI (Python 3.11+), deployed on Render. One main application file defines the HTTP endpoints; the logic lives in backend/services, backend/utils and backend/components.
Agent engine	LangGraph, a compiled graph of nodes with a MongoDB checkpointer that persists state between turns.
Models	Google Gemini. gemini-3.6-flash writes answers and does the heavy reasoning; gemini-3.1-flash-lite handles quick utility steps (intent reading, relevance grading, query rewriting, tool steps). Each is the other's fallback if a call fails. Embeddings: gemini-embedding-001 at 768 dimensions.
Database	MongoDB Atlas: user records, conversations, memory, usage counters, GridFS file storage, LangGraph checkpoints, and Atlas Vector Search over the documents and memory collections.

Authentication	Clerk. The browser sends a Clerk-issued JWT; the backend verifies it on every request. A shared guest sandbox identity exists for the demo.
Other services	browserless.io (remote headless browser over CDP), a WebAssembly sandbox for running Python, GitHub Actions for real test results, Google OAuth for Calendar, Gmail, Drive and Docs, and errAgent for production incident monitoring.

3. How one message travels through the system

1. The browser posts the message to POST /api/chat with the Clerk token. The response is a Server-Sent Events stream, so progress and tokens arrive as they happen.
2. The backend verifies the token, loads the user's record and the knowledge bases they may use, and rejects any request for a scope they are not authorised for.
3. It loads the user's settings (answer mode, deep thinking, target repository, profile), checks the usage budget, and loads the conversation. The last ten messages go into the graph state.
4. The compiled LangGraph runs. A coordinator reads the intent and builds a plan of nodes (section 4). Each node reports progress into the stream; this is what the Execution Trace panel in the app shows.
5. When the graph finishes, one voice composer builds the final prompt: the Sonic Assistant persona, a grounding block for the kind of answer (strict knowledge base, open knowledge base, web, tool output, or conversational), the data, relevant memory, and the conversation history.
6. The answer is streamed token by token from Gemini to the browser.
7. Checks run on the finished answer (section 5 and 9), the conversation is saved, and background tasks store what is worth remembering and record usage.

4. The agent graph (LangGraph)

Every turn enters the graph at the coordinator node. The coordinator runs the reasoner, which asks the small model for a structured reading of the message: what is being asked, which tools might be needed, the user's emotional state, and any safety signals. Rule-based intent classification and a plan builder then decide which nodes run and in what order.

- **retrieve_node** searches the knowledge bases the user may access. **grade_documents_node** asks the small model whether the results are relevant. If not, **rewrite_query_node** rephrases the question and retrieval runs again.
- **conversational_node** handles ordinary conversation with no retrieval or tools.
- **tool_agent_node** runs multi-step tool work (section 7).
- **memory_recall_node** and **memory_save_node** read and write long-term memory (section 8).
- **summarizer_node** and **pr_summary** summarise material, including GitHub pull requests.
- **propose_write_node** and **execute_write_node** split real-world writes into a proposal, an approval by the user, and only then the action.
- **formatter_node** does no model call. It packages whatever the earlier nodes produced and labels the answer type so the voice composer knows which grounding rules apply. **generate_node** ends the graph.

State survives between turns because the graph is compiled with a MongoDB checkpointer keyed by the conversation. Fields meant to last (for example a paused clarification question or a pending approval) are kept; per-turn fields are reset at the start of each turn. A retention job prunes old checkpoints, because an early version let them fill most of the free database quota.

5. Retrieval and knowledge bases

Documents belong to a knowledge base, called an affiliate in the code (for example Sonic lore, Dragon Ball data, Jack's portfolio, a fitness site's guide). Every chunk in the documents collection carries its affiliate tag. A search always includes the user's allowed affiliates as a filter inside the MongoDB $vectorSearch stage, so unauthorised documents are never retrieved at all; there is no after-the-fact filtering. Each user can also have a private personal knowledge base.

The search strategy is chosen from the question. Questions about time or recency ("latest", "when", "timeline") use lexical (keyword) scoring. Comparisons and long questions use hybrid search that fuses vector and lexical rankings. Everything else uses pure vector search.

Three searches in parallel, including the NetworkX knowledge graph

The retrieve node runs three lookups at the same time and merges them: a vector search over the user's allowed knowledge bases (the top three chunks), a search over any documents attached to the current conversation (these are ranked first), and a search of a NetworkX knowledge graph. The graph is a directed graph of entities and the relationships between them. For every entity in the graph whose name appears in the question, the node walks two hops outward and adds the relationships it finds to the context as lines such as "Connection: A --(relation)--> B". This is the GraphRAG part of retrieval: it supplies how things relate, which a similarity search over chunks does not.

Where the graph comes from: when a document is ingested, after its chunks are indexed, a small model (gemini-3.1-flash-lite) reads the text in blocks of about six thousand characters and extracts relationships as subject, relation and target. They are stored in a graph_edges collection in MongoDB, one document per edge, each tagged with its knowledge base and its source file. Retrieval reads only the edges of the knowledge bases the user is allowed to use, so the graph is isolated per tenant exactly like the vector search, and deleting a document removes its edges along with its chunks. Entities are matched as whole names, generic page-title words such as "About" or "Home" are ignored, and at most twelve relationship lines are added to one answer's context. The graph for each knowledge base is cached in the backend for about five minutes, so a newly ingested document's relationships appear within that time.

The graph step is best effort by design. If it fails, the upload still succeeds and retrieval simply has no graph facts for that file. Documents that were indexed before the graph step existed were backfilled with a one-off script that reads their chunks back from the database. The graph is only as good as the extraction: it records what the text states and can miss or oversimplify relationships.

Answer modes: in strict mode the assistant answers only from retrieved documents and says plainly when it cannot find the answer; in open mode it may combine documents with general knowledge. Two safety nets run after generation. If a strict answer says nothing was found, the system rewrites the query and retries once. A separate reward evaluator judges the finished answer for knowledge-base and tool-backed responses and may regenerate it once with feedback; failures are saved as corrections that become guardrails for similar questions later.

6. The ingestion function app: how new documents get in

The folder local_function_app contains function_app.py, the ingestion pipeline. It is called the function app because it was designed in the shape of a serverless function: a self-contained worker that does one job. It is deliberately independent of the main backend. It imports nothing from the backend package and has its own database connection and its own environment loading, and the API runs it as a separate subprocess. A heavy or failing document cannot take the API down.

How it connects to the rest of the system

1. An authorised user uploads a file from the Self Service page. The backend (POST /api/upload) checks that the user may ingest into that knowledge base, then stores the raw file in MongoDB GridFS with metadata: the affiliate, a status of "raw", and processed set to false.
2. The endpoint then launches function_app.py as a subprocess (in a worker thread so the event loop is not blocked) and waits for it to finish.
3. The pipeline finds the files whose status is "raw" (it polls briefly in case the write has not landed yet). For a PDF it extracts each page; for an image it asks a Gemini vision model to describe it. Embedded images in PDFs (up to five per file) are described the same way and their descriptions are folded into the page text, so pictures become searchable.
4. The text is split into overlapping chunks (about 1000 characters with 200 of overlap), embedded with gemini-embedding-001 at 768 dimensions, and written to the documents collection through the Atlas Vector Search index named vector_index, each chunk tagged with its affiliate, source filename and page.
5. Best effort, after the chunks are stored: a small model extracts the relationships between the entities in the text and stores them as knowledge-graph edges for that knowledge base and file (see section 5). If this step fails it is logged and skipped; it never fails the upload.
6. On success the GridFS file's status becomes "pages", which is what the Self Service list shows as indexed. On any failure the pipeline rolls back: the file is deleted from GridFS and its vectors and graph edges are removed, so no half-indexed document is left behind.

The chat side never calls the function app directly. They meet only in MongoDB: the function app writes chunks, and the retrieve node reads them with the same index and the same embedding model. Deleting a document from Self Service removes both the file and its chunks.

7. Tools and the tool agent

When a message needs action (reading a repository, checking a calendar, running code), the coordinator routes it to the tool agent, which loops: decide a step, run a tool, read the result, repeat until it can answer. There are two interchangeable engines. The original ReAct loop writes its decision as structured text each step. The native engine uses real function calling with one growing conversation, which is designed to let the model provider cache the shared prefix and so cost less. A switch selects the engine, and tools the native engine does not yet declare run through the ReAct loop.

- **GitHub:** list a repository's files, read files, search code, compare branches, list commits and pull requests, summarise a PR. Reads use a per-request tarball download of the repo, which is faster than

many separate API calls. Users can add their own GitHub token so work runs under their own access. Creating a pull request or issue always shows an approval card first.

- **Google:** per-user OAuth for Calendar (read, create, update), Gmail (search, read, send) and Drive and Docs (search, read, append to a chosen document). Sends and edits are approval-gated.
- **Python sandbox:** code runs in a real WebAssembly interpreter (CPython compiled to WASI) with no filesystem and no network access, which is a genuine isolation boundary, not a restricted exec().
- **Headless browser and URL reader:** read a page cheaply with a plain fetch and text extraction, or drive a real remote browser through browserless.io when a page needs rendering.
- **Test runs:** the agent can trigger the repo's own GitHub Actions workflow and read the real pass or fail result instead of guessing.
- **Admin-only tools:** database queries (writes need approval) and read-only access to production incidents, logs and deploy status through errAgent. These are hidden from non-admins and refused in code if requested anyway.

Users can press Stop to cancel a running answer, or send a message while the agent is working to steer it; steering is applied at the next step boundary. If the agent needs information it asks a clarifying question, remembers where it was, and resumes after the answer.

8. Memory

Memory is per user and stored in MongoDB. It has two layers. Facts are short statements about the user ("works on a Next.js frontend", "prefers short answers") with a category and a confidence score. When a new fact arrives it is embedded and compared with existing ones; a small model judges whether it duplicates, replaces or merely sits beside an existing fact. Repeats raise confidence, a replacement resets it, and confidence decays when a fact is not seen again. Facts that belong together can be linked, and a linked group is recalled together. Conversation notes are embedded summaries of earlier exchanges, searched with Atlas Vector Search.

On every turn a passive semantic lookup injects only the memory that clears a relevance threshold, so most turns add nothing. Periodic compaction merges and prunes memory so it stays small. The Memory page lets the user read, edit, unlink and delete everything that is stored. Other memory-driven behaviours include noticing recurring patterns, open loops (asking afterwards how an upcoming interview or trip went), optional Message First conversation openers, and goal check-ins when a project goes quiet. Asked what it plans to follow up on, Sonic answers from those same stored records for that person only, and describes them only as things it may raise when the person next opens a conversation, never as background activity or messages sent in between. Guests have no private memory.

9. Safety, emotional attunement and character

The reasoner reads the user's emotional state each turn. The state carries across topic changes and decays over time, so after a rough moment the assistant does not turn cheerful just because the subject changed. A separate safety layer watches for signs of crisis, combining a language-pattern check with the model's own reading and taking the higher of the two; while it is active the assistant always switches to a conversational reply instead of a knowledge-base refusal or a data dump. Its approach is a support ladder: it works through the people in the user's life first (family, friends, wider circle, professionals) and offers a

hotline when it is due or when there is imminent danger. Practical requests below the highest level may still keep tool results, so the assistant stays useful.

The assistant has a written profile (tastes, manner, a dry sense of humour), a curated changelog of what has changed about it, counters and approved self-observations that let it state its own track record truthfully, and a small reading shelf of pieces it has read. Rules keep this honest: it does not claim lived experience, it does not claim to have done things it has not done, and anything it says about itself must come from the changelog or measured data.

10. Cost control and usage limits

A budget guard meters every model call in dollars. As of October 2026 there is a global daily cap of five dollars and a per-user allowance of one dollar a day plus a rolling five-hour limit, applied to admins too so testing cannot use up what everyone shares. The system degrades in steps rather than failing suddenly: a notice appears as spending nears a limit; at the limit a low-power mode starts, in which every optional extra pauses together (deep thinking, the reading shelf, the answer re-check) and tool runs get fewer steps; past the block line the assistant says plainly that the limit has been reached and when it resets. The usage page shows a ring with the percentage used. Evaluation runs are tagged so they are excluded from the guard but still counted as real spend.

11. Authentication, roles and multi-tenancy

Identity comes from Clerk. The backend verifies the JWT, maps it to a directory record, and derives the user's groups. Groups decide which knowledge bases the user may read and which they may ingest into; a personal knowledge base is made of two groups, one for reading and one for ingesting. Administrators get extra tools and the Sonic Health dashboard (usage, safety statistics, the assistant's self-observations, the reading shelf, and operational suggestions). The guest sandbox uses a shared identity with locked-down defaults, and an embedded guest identity serves the chat widget used inside a partner fitness site. CORS is restricted to the production domain, preview deployments and local development.

12. The frontend

Pages: Chat (with a collapsible rail of conversations and a right-hand Execution Trace blade), Self Service (upload and manage documents), Tidy Up (plan the sorting of a messy local folder), Memory, Integrations (Google and GitHub), Account (profile and settings), and Sonic Health for admins. Replies stream in as they are written. Citations link to the source document and page and can be downloaded. Photos and files can be attached to a message.

The local folder feature uses the browser's File System Access API. The user picks a folder; the browser uploads a read-only snapshot of its text files (and a numbered text outline of Word documents) to the server so the assistant sees the real working tree, uncommitted changes included. The assistant never writes to disk itself. It proposes edits, the server validates them against the snapshot, and the user reviews a diff card and presses Apply; the browser does the writing and keeps an undo copy. New Word documents and edited copies of existing ones are produced the same way, with checks that the original structure is preserved and the original file is never altered.

13. Reliability and observability

- **Model resilience:** each model has a fallback model, and timeouts, 503s and 504s are treated as transient so one slow call does not lose the turn.
- **Event loop discipline:** database and file work is moved off the async event loop, and a watchdog logs when the loop is blocked and what was waiting. Cold starts on the free hosting tier are the main source of a slow first request.
- **errAgent:** unhandled errors and structured logs are reported to a separate incident-monitoring service, which analyses root causes and can open fix pull requests. Several production bugs in this project were fixed that way.
- **Log hygiene:** user message content is kept out of logs and error reports, enforced by an automated test and a redaction filter.
- **Routines:** a short list of jobs the server runs on a schedule and records: a health check (its own incidents, the latest deploy, failed chat turns, knowledge-graph gaps and the usage budget, with no model call), the reading-shelf refresh, checkpoint pruning, and attachment retention (photos of personal documents do not live forever). Each run is written down with a few counts. Asked what it has planned or what it did, Sonic answers only from this schedule and these records, in the person's own timezone, and says nothing about being awake or tired; administrators also see what the health check found on the Sonic Health page.

14. Testing, evaluations and CI

The backend has over three thousand five hundred automated tests. A GitHub Actions workflow runs them on pull requests, and the tool agent can read that workflow's real results. Separate evaluation suites run real model calls for things unit tests cannot judge: character and voice consistency, tool-use behaviour compared across both engines, and memory quality. Because evaluations cost real money, they are run selectively. Changes are built from failures actually observed rather than guessed at.

15. History: how it got here

- **June 2026:** first commit. A RAG question-answering app over documents, with a React interface and saved conversations.
- **July 2026:** LangChain replaced by LangGraph with entity-graph enrichment and lexical and hybrid search. The multi-agent design (reasoner, coordinator, formatter, summarizer, memory nodes) shipped as v0.8 on July 8. Clerk authentication, deployment to Vercel and Render, the custom domain, web search, a code-interpreter node, approvals before write actions, and streamed thinking in the trace panel followed.
- **August 2026:** an embeddable widget for a partner fitness site, errAgent integration for production incidents, the first CI test workflow, follow-up suggestions.
- **September 2026:** long-term memory and conversation patterns, passive memory recall, repository tools, deep-thinking mode, a headless browser, Google integrations, a stop button.
- **October 2026:** local folder access with diffs and undo, mid-run steering, users' own GitHub tokens, a URL reader, the safety layer, emotional attunement, profile and identity, proactive openers and open loops, the budget guard, a memory upgrade and memory linking, an app shell redesign with the

Patchy mascot, curiosity notes and the reading shelf, per-user spending limits, Word document creation and editing, and a redesigned landing page. An earlier set of built-in productivity modules (notes, task board, calendar module, insights dashboard) was retired in favour of real integrations.

16. Weaknesses, limits and honest caveats

What Sonic Assistant is not good at, and where it can fall short:

- Hosting uses free or low-cost tiers, so the first request after idle time can be slow while the server wakes.
- The assistant has no existence between conversations; what feels like continuity is stored memory and a profile.
- The local folder feature needs a desktop Chromium-based browser (Chrome or Edge) because of the File System Access API.
- Answers depend on what is in the knowledge base the user can see; in strict mode it will say it cannot find something rather than guess.
- The ingestion pipeline handles PDFs and images; other file types are not indexed into knowledge bases.
- The knowledge graph is built by a small model from the text, so it can miss or oversimplify relationships, and a very long document is sampled within a model-call and time budget rather than read to the last page.

17. Frequently asked plumbing questions

What does Sonic do on its own when nobody is chatting?

It does not exist as a presence between conversations, but the server runs a few real jobs on a schedule: a health check every six hours (incidents, the latest deploy, failed chat turns, knowledge-graph gaps, the usage budget), a reading refresh about twice a day, and daily clean-ups of old conversation checkpoints and old attachments. Every run is recorded, and when asked what it has planned or what it did, Sonic answers from the schedule and those records only.

Where does NetworkX fit, and is the knowledge graph actually used?

Yes. NetworkX holds an entity-relationship graph that is queried at retrieval time. The retrieve node runs vector search, conversation-attachment search and a graph lookup in parallel; the graph lookup returns two-hop relationships for entities named in the question and adds them to the context as extra lines. The edges are built when a document is ingested (a small model extracts them from the chunks) and stored in MongoDB per knowledge base, so the graph is tenant-isolated like everything else. LangGraph, not NetworkX, orchestrates the agent steps.

How does the function app connect to the rest of the app?

Through MongoDB and a subprocess call, nothing else. The API stores the upload in GridFS and starts function_app.py as a separate process; that process turns the file into embedded chunks in the documents collection. The chat path later reads those chunks through Atlas Vector Search.

Where does the data live?

Everything is in one MongoDB Atlas database: users and groups, conversations, memory facts and notes, usage counters, uploaded files in GridFS, searchable chunks with embeddings, and LangGraph checkpoints.

How is one customer's data kept from another customer's? (tenant isolation)

The check happens inside the database query, not afterwards. The user's verified Clerk identity is mapped to groups, the groups to the knowledge bases (affiliates) they may read, and that list is passed as a filter in the MongoDB $vectorSearch stage, so a document from another tenant is never returned to the application at all. Requests for a scope the user does not hold are rejected with a 403. Personal knowledge bases and memory are keyed to the individual user.

What happens between uploading a PDF and being able to ask about it?

Upload to Self Service stores the raw file in MongoDB GridFS marked "raw". The API then runs the ingestion function app as a subprocess. It extracts the pages (describing any images with a vision model), splits the text into chunks of about 1000 characters, embeds each chunk with gemini-embedding-001, and writes the chunks to the documents collection, which Atlas Vector Search indexes as vector_index. The file is then marked "pages" and appears as indexed. From that moment the retrieve node can find it.

What if ingestion fails halfway through?

It rolls back. The pipeline deletes the uploaded file from GridFS and removes any chunks it already wrote for it, so the database is never left with a half-indexed document, and the upload reports the failure.

What is the actual flow of a chat message?

The browser sends the message and the Clerk token to POST /api/chat. FastAPI verifies the token and the user's knowledge-base scope, loads settings, memory and the last ten messages, and runs the LangGraph: the coordinator reads the intent and plans, then the retriever, the tool agent or the conversational node does the work. A voice composer builds one prompt, Gemini streams the answer back to the browser over Server-Sent Events, post-checks run, the conversation is saved, and memory is updated in the background.

How does memory avoid storing or recalling junk?

New facts are compared with existing ones so duplicates reinforce instead of piling up, memory is recalled only when it clears a relevance threshold (so most turns add none), unused facts decay, compaction merges and prunes, and the user can review, edit and delete everything on the Memory page.

What are its weaknesses?

The first request after idle time can be slow because the hosting is on a free or low-cost tier and cold-starts. The assistant exists only while a conversation is running, and its sense of continuity is stored memory. It indexes only PDFs and images into knowledge bases, and the local folder feature needs desktop Chrome or Edge. In strict mode it will say it cannot find an answer rather than guess, and its spending is capped daily per user and overall.

What decides whether a question searches the knowledge base, calls a tool, or just chats?

The coordinator. A small-model reasoner reads the message, rules classify the intent, and a plan builder chooses and orders the nodes.

Which models does it use, and what happens when one fails?

gemini-3.6-flash for answers and deep reasoning, gemini-3.1-flash-lite for utility steps, and each is the other's fallback. Transient errors are retried or routed to the fallback.

Can it change things on its own?

No. Every write (pull request, issue, email, calendar change, document edit, database write) is proposed, shown to the user, and executed only after approval. Local files are only ever written by the user's own browser.

How did the project start, how has it evolved, and what was removed along the way?

It started in June 2026 as a document question-answering app, became a LangGraph multi-agent system in July (v0.8 on July 8), gained long-term memory and tools through September, and in October gained local folder access, a safety layer, spending limits and Word document editing. An early set of built-in productivity modules (notes, task board, calendar module and insights dashboard) was retired in favour of real integrations such as Google Calendar and GitHub.

How does it know what it is and what has changed about it?

From this reference, a written profile, and a curated changelog of capabilities. It is told to answer questions about itself only from those and from measured data, never from invention.